VendorFlow: A Unified SaaS Platform for Local Grocery & Pharmacy Digitization

Project Proposal for UCS503P

Submitted to: Dr. Jeelani
Thapar Institute of Engineering and Technology
August 16, 2026

Project Team
Pranjal Chitravanshi Kunal Thakur Mannat Saini

1 Higher-order Goal

This project aims to strengthen local retail resilience by giving small grocery stores and pharmacies an affordable path to digital commerce. While the broader vision is economic empowerment of neighborhood businesses, the immediate engineering objective is to translate reliable order fulfillment and vendor coordination into a measurable, deployable software solution.

2 Time-to-value

- We will deliver meaningful increments quickly by prototyping the core ordering and vendor-routing workflow and validating it with a small set of pilot vendors within a short iteration window.
- We will prioritize fast feedback over perfection by using weekly iterations and targeting CI-backed automation early to reduce release friction.
- Success is defined by what can be delivered and measured in the first iteration, then improved based on vendor and customer feedback.

3 Problem Statement

Small grocery stores and pharmacies in most Indian cities and towns rely on phone calls, WhatsApp, spreadsheets, or paper records to manage orders and inventory. Common pain points include:

- **No online presence:** Most vendors cannot afford a custom website, app, or delivery infrastructure of their own.
- **Manual, error-prone operations:** Inventory and order tracking done by hand leads to stockouts, missed orders, and delivery mix-ups.
- **Fragmented discovery:** Customers have no single place to find nearby stores, compare stock, or track deliveries.
- **Regulatory friction for pharmacies:** Prescription-backed orders need a verification step that ad-hoc channels such as calls and chat apps do not support reliably.

Consumer demand for online ordering continues to grow, but the smallest vendors are the ones least able to build this infrastructure themselves. A shared platform lets many small businesses digitize at once without each bearing the full engineering cost.

4 Proposed Solution

4.1 Overview

VendorFlow is proposed as a full-stack SaaS platform with the following components:

- **Customer portal:** Discover nearby grocery/pharmacy vendors, browse products, upload prescriptions where required, place orders, and track deliveries.
- **Vendor portal:** Vendor onboarding and approval workflow, inventory management, order management, and reliability/analytics dashboard.
- **Delivery partner portal:** Accept assigned deliveries and update status in real time.
- **Admin portal:** Vendor approval, prescription verification oversight, platform analytics, and audit logs.
- **Smart vendor routing:** Route an order to the best-fit vendor using distance, live stock, and a computed reliability score.
- **Credit system:** An in-platform credit/wallet mechanism for customers and vendors, supporting promotional credits, refunds, and settlement between platform and vendors.

4.2 Core Workflow

1. The customer browses nearby vendors and adds items to the cart, uploading a prescription for pharmacy items where required.
2. The system applies smart vendor routing using distance, stock, and reliability score. For pharmacy orders, the prescription is queued for manual verification.
3. The order is confirmed, a delivery partner is assigned, and status updates flow through the lifecycle: **Placed** $\rightarrow$ **Confirmed** $\rightarrow$ **Out for Delivery** $\rightarrow$ **Delivered**.
4. The customer and vendor receive real-time notifications at each stage. Credit/wallet balances are updated on completion, refund, or promotional events.

4.3 Operational Constraints

- Keep customer- and vendor-facing interfaces usable on low-to-mid range devices through responsive web design.
- Keep order-matching and routing logic heuristic and explainable rather than research-grade, focusing on correctness, latency, and auditability.
- Design the backend for high throughput because order placement, routing, and notification delivery are on the platform's critical path.

5 Solution Approach

This is an engineering course project, so the focus is on system architecture, performance, and correctness of the transactional workflow rather than novel algorithm research.

5.1 Performance-focused Backend

- **C++:** High-throughput backend services including order intake, routing engine, and inventory locking for predictable low-latency performance under load.
- **Python:** Auxiliary services such as analytics jobs, reliability-score recomputation, and admin/reporting tooling.

- **JavaScript:** Customer, vendor, delivery, and admin front-ends, together with lightweight orchestration/API-gateway glue where appropriate.
- **Vendor routing:** A simple weighted-score heuristic based on distance, stock availability, and reliability score, without claiming state-of-the-art optimization.

5.2 System Architecture

The proposed system follows a multi-tier architecture:

- **Frontend:** Responsive web applications for customer, vendor, delivery partner, and admin portals, built in JavaScript.
- **High-performance backend:** C++ REST/WebSocket services for authentication, order lifecycle, vendor routing, inventory locking, and the credit/wallet ledger.
- **Auxiliary services:** Python background jobs for reliability-score updates, routing-timeout handling, notification queue processing, and analytics pipelines.
- **Cache/queue layer:** Redis for caching, rate limiting, session data, and background job/notification queues.
- **Primary database:** PostgreSQL for users, vendors, products, inventory, orders, credit/wallet ledger, reviews, and audit logs.
- **Real-time layer:** Socket.IO or an equivalent WebSocket layer for real-time order and delivery status notifications.

5.3 CI/CD

CI/CD will be used to:

- automatically build and run C++ unit tests, Python service tests, and frontend tests on every change;
- produce repeatable deployment artifacts to staging for each service; and
- enable safe and frequent releases of incremental features across the portals.

6 Evaluation Criteria

Success will be evaluated using measurable metrics tied to the core order-fulfillment workflow.

6.1 Primary Evaluation Metric

Order-to-confirmation latency (OCL) is the time between order placement and vendor confirmation after routing.

- **Target:** Median OCL ≤ 60 seconds during the pilot window.
- **Attribution:** Measured using backend timestamps from the order-placed event to the vendor-confirmed event.

6.2 Secondary Evaluation Metrics

- Delivery success rate: percentage of orders marked Delivered without escalation or cancellation.
- Vendor reliability accuracy: correlation between the computed reliability score and actual on-time fulfillment during the pilot.

- Prescription verification turnaround: median time from upload to admin-verified status for pharmacy orders.
- System reliability: availability of login, ordering, and routing during business hours, with an example pilot target of at least 99% uptime.
- Backend throughput: sustained requests per second served by the C++ order and routing services under load-test conditions.

6.3 Pilot Validation Plan

The pilot will recruit approximately 10–20 customers and 3–5 vendor accounts, with a mix of grocery and pharmacy vendors. Where real vendors are unavailable, simulated vendors may be used. Metrics will be compared before and after implementation using short interviews or existing process observations to estimate baseline manual-process timings.

7 Scalability

The system should scale both theoretically and practically.

7.1 Theoretical Scalability

The system should support growth in vendors, orders, and concurrent portal usage by:

- decoupling the C++ backend, Python auxiliary services, and frontends;
- enabling pagination, indexing, and read replicas for common PostgreSQL queries;
- using Redis for caching and rate limiting to shield the primary database; and
- designing backend APIs to be stateless where possible.

7.2 Operational Deployability

The system should run on common infrastructure using:

- managed PostgreSQL and Redis instances;
- containerized deployment for the C++ backend, Python services, and frontends; and
- a CI/CD pipeline for staging and production across all services.

8 Technology and Engine Availability

The project will use mature technologies and services rather than inventing new infrastructure:

- High-performance C++ web/service framework for the core backend.
- Python framework for auxiliary/background services and analytics.
- JavaScript framework for the four frontend portals.
- PostgreSQL as the managed relational database.
- Redis for caching, rate limiting, and job/notification queues.
- JWT-based authentication with role-based access control.
- Object storage for prescription images and product photos.
- Unit and integration testing frameworks across C++, Python, and JavaScript.

- CI runner and staging deployment target.

9 Project Scope and Deliverables

9.1 Initial Deliverable

- Customer, Vendor, Delivery Partner, and Admin portals with JWT authentication and role-based access control.
- Vendor onboarding and approval workflow with product and inventory management.
- Smart vendor routing based on distance, stock, and reliability implemented in the C++ backend.
- Shopping cart, checkout, and order lifecycle management.
- Prescription upload with manual admin verification.
- Delivery assignment and status-based tracking.
- Basic logging and timestamps for OCL measurement.
- CI with builds, C++/Python backend unit tests, and linting.
- CD to staging with a single pipeline job.

9.2 Subsequent Deliverables

- Credit/wallet system for customers and vendors, including promotional credits, refunds, and settlement.
- Real-time notifications through Socket.IO, together with email and in-app notifications.
- Reviews and ratings feeding into the vendor reliability score.
- Vendor and admin analytics dashboards.
- Redis-backed caching and rate limiting hardened for scale.
- Background jobs for notification queueing, routing timeouts, and reliability-score recomputation.
- Audit logs and activity history across vendor and admin actions.
- Improved search/filtering and indexed queries for scale readiness.

10 Risks and Mitigations

- **Data privacy and consent:** Minimize collected data, especially prescription images; store only what is needed and restrict access to verified administrators.
- **Vendor adoption:** Keep the vendor portal simple, with minimal steps for onboarding, inventory updates, and order confirmation.
- **Backend complexity:** Mitigate C++ complexity through strong test coverage, clear service boundaries, and CI-enforced builds before merge.
- **Evaluation measurement ambiguity:** Instrument timestamps and define clear order-lifecycle states before development begins.
- **Operational issues during the pilot:** Use staging early and ensure CI/CD produces repeatable deployments for all services.

11 Timeline

Phase	Major Tasks	Expected Deliverable
Weeks 1–2	Requirements, architecture, database design, and UI planning	System requirements and architecture design
Weeks 3–4	Authentication, role-based access, vendor onboarding, and inventory	Initial customer/vendor/admin foundation
Weeks 5–6	Cart, checkout, order lifecycle, and smart vendor routing	Core ordering and routing workflow
Weeks 7–8	Prescription upload, manual verification, and delivery assignment	Pharmacy and delivery workflow
Weeks 9–10	Real-time status, Redis queues/caching, wallet/credit functionality	Integrated transactional workflow
Weeks 11–12	Analytics, reviews, testing, CI/CD, load testing, and refinement	Validated prototype and staging deployment

12 Conclusion

VendorFlow is a deployable SaaS platform intended to help small grocery stores and pharmacies digitize inventory, ordering, delivery, and vendor management. The proposed architecture uses a high-performance C++ backend, Python auxiliary services, PostgreSQL and Redis for storage and caching, and JavaScript frontends across four portals.

The project emphasizes time-to-value through rapid iterations, CI/CD for frequent and safe releases, and measurable evaluation criteria, particularly order-to-confirmation latency. The focus remains on engineering workflow, backend performance, correctness, and scalability readiness rather than research-driven novelty.